OurMocks

2027 CYCLE

MOCK · PRACTICE · CRACK

CUET UG

Computer Science

Queue & Deque Data Structures

FIFO Architecture, Circular Buffers, Modulo Arithmetic, Deque Types & Priority Queues

INSIDE
Concept Notes

INSIDE
Solved Examples

INSIDE
Practice MCQs + Solutions

Aligned to the latest official syllabus · Concept-first explanations

www.ourmocks.com

Syllabus Alignment

Section / Unit	Official NTA Syllabus Pointers	Core Focus & High-Yield Exam Traps
Section B1: Topic 2 (Class 12)	Queue ADT & FIFO Architecture: Linear Queue, Front and Rear pointers, operations (`enqueue`, `dequeue`, `peek`), underflow and overflow conditions.	FIFO (First-In, First-Out) discipline. Linear queue pointer drift causing "False Overflow" when Rear $=$ Max -1 despite available space.
Section B1: Topic 2 (Class 12)	Circular Queues & Modulo Arithmetic: Ring buffers, pointer wrap-around mechanics, full queue condition, element count formulas.	Modulo wrap equations: ptr $=$ (ptr $+$ 1)% Max. Full condition: ((Rear $+$ 1)% Max) $==$ Front. Empty condition: Front $=-1$.
Section B1: Topic 2 (Class 12)	Deque Variations & Implementations: Double-Ended Queue (Deque), input-restricted and output-restricted deques, Python `collections.deque` vs `list`.	Performance trap of `list.pop(0)` ($O(N)$ linear time). True $O(1)$ operations via `collections.deque`. Distinction between input and output restrictions.

EXAM PATTERN

CUET UG Exam Architecture: Unit 3 (Queue) accounts for approximately 8% to 12% of Section B1 in the CUET UG Computer Science paper (4 to 5 MCQs). Frequent question archetypes include computing circular queue pointer values after a sequence of enqueues and dequeues, determining the time complexity of queue implementations, distinguishing between input-restricted and output-restricted deques, and tracing BFS queue operations.

Concept Module 1: The Queue Abstract Data Type & FIFO Principle

1.1 First-In, First-Out (FIFO) Architecture

A **Queue** is a linear data structure governed strictly by the **First-In, First-Out (FIFO)** access discipline. In a queue:

- Elements enter the structure from one logical boundary, designated as the **Rear (or Tail)**, through the operation known as **Enqueue**.
- Elements depart from the opposite logical boundary, designated as the **Front (or Head)**, through the operation known as **Dequeue**.
- Just as in a real-world ticket counter, the element that arrives earliest is the first element serviced and removed.
- A Queue is an **Abstract Data Type (ADT)** whose public contract guarantees FIFO ordering regardless of whether the internal implementation uses contiguous arrays, circular ring buffers, doubly-linked blocks, or heap trees.

1.2 Queue Pointer Invariants & State Signatures

In an array-based queue of capacity Max:

- **Empty Queue (Underflow):** Front $= -1$ and Rear $= -1$ (or alternatively, in dynamic tracking, Front $>$ Rear). Attempting a `dequeue()` or `peek()` in this state raises **Queue Underflow**.
- **Single-Element Queue:** Front $=$ Rear $\neq -1$. When this solitary element is dequeued, both pointers must be reset to -1 to represent the empty state.
- **Full Queue (Overflow):** Occurs on bounded queues when the underlying buffer cannot accept additional elements without displacing unread items.

1.3 The Linear Queue Memory Drift Hazard (False Overflow)

In a naive linear array implementation of a queue:

1. Every `enqueue` increments Rear (Rear $\rightarrow$ Rear $+ 1$).
2. Every `dequeue` increments Front (Front $\rightarrow$ Front $+ 1$).
3. As items are dequeued, the physical slots at the beginning of the array $(0, 1, ..., \text{Front} - 1)$ become permanently vacant and un-utilizable.
4. Eventually, Rear reaches the terminal index Max $- 1$. When the next `enqueue` arrives, the system declares **Queue Overflow**, even if slots 0 to Front $- 1$ are completely empty!
5. This crippling limitation is termed **False Overflow** or **Memory Drift**. It is the fundamental motivation for developing **Circular Queues**.

Concept Module 2: Queue Primitive Operations & Complexity

2.1 The Canonical Primitive Operations

The standard Queue ADT contract specifies five core operations:

1. `enqueue(item)`: Appends an element to the Rear of the queue. If capacity is bounded and full, raises Queue Overflow. Moves Rear $\rightarrow$ Rear $+ 1$ ($O(1)$).
2. `dequeue()`: Removes and returns the element residing at the Front of the queue. If empty, raises Queue Underflow. Moves Front $\rightarrow$ Front $+ 1$ ($O(1)$ in circular/linked queues).
3. `peek()` / `front()`: Returns the element at the Front of the queue **without** mutation. Raises Underflow if empty ($O(1)$).
4. `is_empty()`: Returns Boolean `True` if Front $= -1$ or the element count is 0 ($O(1)$).
5. `size()`: Returns the total number of active items stored in the queue ($O(1)$).

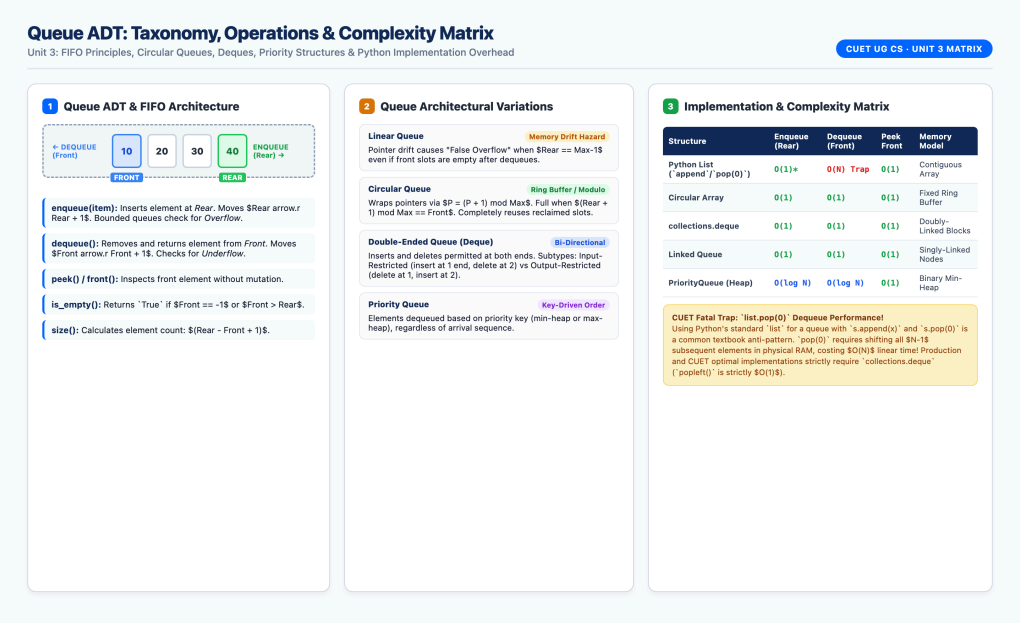


Figure 1: Figure 1: Queue Types, Operations & Complexity Matrix: Structural taxonomy comparing Linear Queues, Circular Buffers, Deques, and Priority Queues with their time and memory overheads.

2.2 Time & Space Complexity Invariants

- **Optimal Implementations:** Under circular arrays, `collections.deque`, or linked lists, all primitive operations (`enqueue`, `dequeue`, `peek`, `is_empty`, `size`) execute in strictly $O(1)$ constant time.
- **Linear Array List Implementations:** In a Python list using `list.pop(0)`, `dequeue` degrades to $O(N)$ linear time because physical memory must shift all $N-1$ subsequent elements.

Concept Module 3: Circular Queue & Modulo Arithmetic

3.1 Ring Buffer Geometry

A **Circular Queue** eliminates the memory drift problem of linear queues by conceptually connecting the last slot of the array (index $= \text{Max} - 1$) back to the first slot (index $= 0$), forming a continuous **Ring Buffer**.

- Both Front and Rear pointers advance clockwise around the circular track using **Modulo Arithmetic**.
- Whenever a pointer exceeds $\text{Max} - 1$, modulo division wraps it back to 0:

$$\text{ptr} \rightarrow (\text{ptr} + 1) \% \text{ Max}$$

3.2 Formal Mathematical Invariants of Circular Queues

For a circular queue of fixed maximum capacity Max:

1. **Initialization (Empty State):**
$$\text{Front} = -1, \quad \text{Rear} = -1$$
2. **Enqueue Pointer Progression:**
   - If queue is empty (Front $= -1$): set Front $= 0$, Rear $= 0$.
   - Otherwise: set Rear $= (\text{Rear} + 1) \%$ Max.
3. **Dequeue Pointer Progression:**
   - If queue has exactly one element (Front $=$ Rear): reset Front $= -1$, Rear $= -1$.
   - Otherwise: set Front $= (\text{Front} + 1) \%$ Max.
4. **Queue Full Condition (Overflow):** The circular queue is full when advancing Rear by one position would collide with Front:
$$(\text{Rear} + 1) \% \text{ Max} == \text{Front}$$
5. **Current Element Count Formula:**
$$\text{Count} = (\text{Rear} - \text{Front} + \text{Max}) \% \text{ Max} + 1 \quad \text{(when not empty)}$$

Concept Module 4: Double-Ended Queue (Deque) Architecture

4.1 Bi-Directional Operational Specification

A **Double-Ended Queue (Deque)** (pronounced "deck") is a generalized linear data structure where insertions and deletions are permitted at **both** logical ends (Front and Rear).

- Supported Operations:
  1. `append(item)` / `insert_rear(item)`: Inserts element at the Rear ($O(1)$).
  2. `appendleft(item)` / `insert_front(item)`: Inserts element at the Front ($O(1)$).
  3. `pop()` / `delete_rear()`: Removes and returns element from the Rear ($O(1)$).
  4. `popleft()` / `delete_front()`: Removes and returns element from the Front ($O(1)$).
  5. `peek_front()` and `peek_rear()`: Inspects terminal elements ($O(1)$).

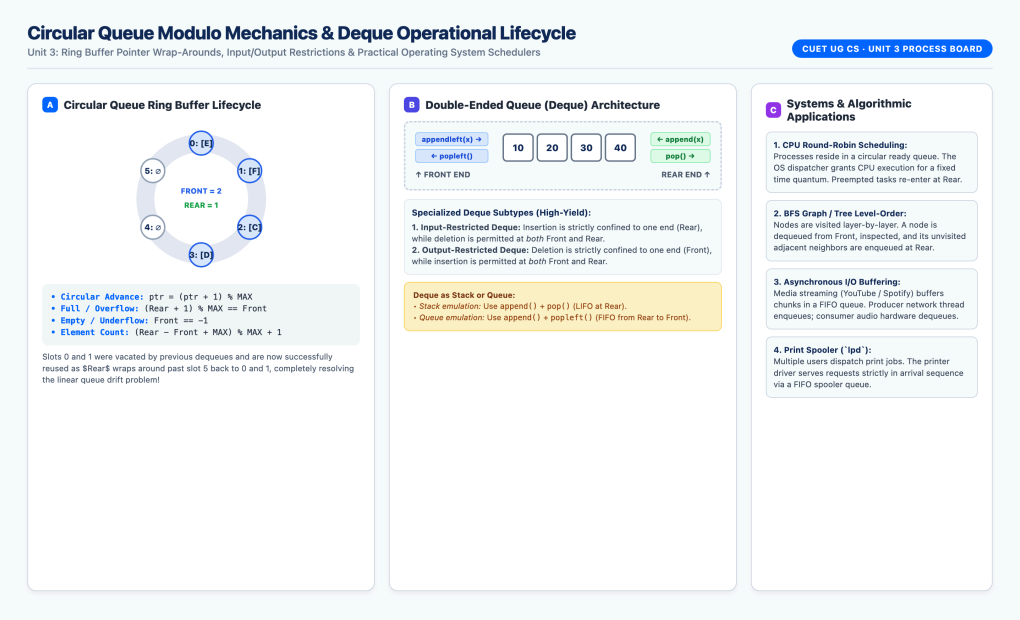


Figure 2: Figure 2: Circular Queue Modulo Mechanics & Deque Operational Lifecycle: Ring buffer pointer wrap-around tracking, specialized input/output restrictions, and real-world system schedulers.

4.2 Specialized Deque Subtypes (High-Yield CUET Topic)

To enforce specific access constraints, computer scientists define two restricted variants of the deque:

1. **Input-Restricted Deque:**
   - **Insertion:** Restricted strictly to **one end** (typically the Rear).
   - **Deletion:** Permitted freely at **both ends** (Front and Rear).
2. **Output-Restricted Deque:**
   - **Deletion:** Restricted strictly to **one end** (typically the Front).
   - **Insertion:** Permitted freely at **both ends** (Front and Rear).

4.3 Universal Data Structure Emulation

A Deque is a universal linear container that natively emulates both standard stacks and standard queues:

- **Stack Emulation (LIFO):** Restrict operations to one end: `append()` to push and `pop()` to pop (or `appendleft()` and `popleft()`).
- **Queue Emulation (FIFO):** Use opposite ends: `append()` to enqueue at Rear and `popleft()` to dequeue from Front.

Concept Module 5: Priority Queues & Heap Ordering

5.1 Priority-Based Ordering vs Chronological Arrival

In a **Priority Queue**, elements are serviced not strictly by arrival timestamp, but according to an associated **Priority Key**:

- An item with higher priority is dequeued before an item with lower priority.
- If two elements possess identical priorities, they are serviced according to standard FIFO arrival order.
- Two standard configurations exist:
  1. **Min-Priority Queue:** The element with the **smallest** numeric key has highest priority (e.g. Dijkstra's shortest path algorithm, hospital triage priority 1 = critical).
  2. **Max-Priority Queue:** The element with the **largest** numeric key has highest priority (e.g. CPU priority task schedulers).

5.2 Implementation Structures & Complexity

- **Unsorted List:** Enqueue is $O(1)$, but Dequeue requires searching for the minimum element, taking $O(N)$ linear time.
- **Sorted List:** Enqueue requires finding the insertion point, taking $O(N)$ time, while Dequeue is $O(1)$.
- **Binary Heap (Optimal):** Both `enqueue` (heap push) and `dequeue` (heap pop) execute in strictly $O(\log N)$ logarithmic time. Python implements this via the standard `heapq` module.

Concept Module 6: Python Implementation Paradigms

6.1 The Python List Anti-Pattern (`pop(0)` Hazard)

Textbooks frequently present simple queues implemented using standard Python lists:

```
class ListQueue:
    def __init__(self):
        self._items = []
    def enqueue(self, item):
        self._items.append(item)   # O(1) amortized
    def dequeue(self):
        if not self._items:
            raise IndexError("Queue Underflow")
        return self._items.pop(0)  # O(N) linear time catastrophe!
```

- Because CPython lists are contiguous pointer arrays, deleting at index 0 requires the runtime to shift all remaining $N-1$ memory pointers one slot leftward via `memmove()`.
- Performing N dequeues on an N-element list incurs $O(N^2)$ quadratic total work!

6.2 The Production Standard: `collections.deque`

Python's standard library provides `collections.deque`, specifically optimized for fast two-ended operations:

- Deque is internally implemented as a doubly-linked list of 64-element memory blocks.
- `append()` and `popleft()` operate strictly in $O(1)$ worst-case time with zero contiguous array reallocation.

```
from collections import deque

class FastQueue:
    def __init__(self):
        self._items = deque()
    def enqueue(self, item):
        self._items.append(item)     # strictly O(1)
    def dequeue(self):
        if not self._items:
            raise IndexError("Queue Underflow")
        return self._items.popleft() # strictly O(1)
```

6.3 Synchronized Multi-Threaded Queue (`queue.Queue`)

For concurrent worker thread pipelines:

- `queue.Queue(maxsize=0)` provides thread-safe FIFO access with mutex locks and condition variables (`put()`, `get()`, `task_done()`, `join()`).
- Blocks calling threads when attempting to `get()` from an empty queue or `put()` to a full bounded queue, coordinating producer-consumer workflows safely.

Concept Module 7: Systems & Algorithmic Applications

7.1 Operating System Process Scheduling

- **First-Come, First-Served (FCFS):** Non-preemptive scheduling where CPU processes are executed strictly in arrival order from a FIFO ready queue.

- **Round-Robin (RR) Preemptive Scheduling:** Processes are placed in a circular ready queue. The OS gives each task a fixed time slice (quantum). If a process does not complete within the quantum, it is preempted, moved from Front, and enqueued at Rear.

7.2 Breadth-First Search (BFS) in Graphs and Trees

Unlike Depth-First Search (which uses a Stack / recursion), Breadth-First Search systematically traverses graphs layer-by-layer using a FIFO Queue:

1. Enqueue root node.
2. While queue is not empty:
   - Dequeue front node u and process it.
   - For every unvisited neighbor v of u, mark v as visited and enqueue v.

- BFS guarantees discovering the shortest path in unweighted networks.

7.3 Asynchronous Hardware & Software Buffering

- **Print Spooler:** Multiple print requests submitted across a network are stored in a spooler queue, serviced in chronological order by the printer hardware.
- **Keyboard Buffer:** Keystrokes typed while the CPU is busy are placed in an OS FIFO circular keyboard buffer so no inputs are dropped.
- **Multimedia Streaming:** Audio and video players buffer downloaded media chunks in a FIFO queue to prevent playback stutter during network throughput fluctuations.

Concept Module 8: Comprehensive Worked Examples

WORKED EXAMPLE 1: CIRCULAR QUEUE MODULO STATE TRACING

Consider a circular queue of capacity $\text{Max} = 5$, initially empty ($\text{Front} = -1, \text{Rear} = -1$). Trace the exact values of Front and Rear pointers after each of the following sequential operations:

1. `enqueue(10)`, `enqueue(20)`, `enqueue(30)`
2. `dequeue()`
3. `enqueue(40)`, `enqueue(50)`
4. `dequeue()`, `dequeue()`
5. `enqueue(60)`, `enqueue(70)`

Step-by-Step Pointer Calculation:

- Initial: $\text{Front} = -1, \text{Rear} = -1$.
- Operation 1: First insert sets $\text{Front} = 0, \text{Rear} = 0$. Next two inserts advance Rear:
  - Enqueue 20 → $\text{Rear} = (0+1)\%5 = 1$.
  - Enqueue 30 → $\text{Rear} = (1+1)\%5 = 2$.
  - State: $\text{Front} = 0, \text{Rear} = 2$, Elements: `[10, 20, 30]`.
- Operation 2: `dequeue()` removes 10. Front advances:
  - $\text{Front} = (0+1)\%5 = 1$.
  - State: $\text{Front} = 1, \text{Rear} = 2$, Elements: `[_, 20, 30]`.
- Operation 3: Enqueue 40, 50:
  - Enqueue 40 → $\text{Rear} = (2+1)\%5 = 3$.
  - Enqueue 50 → $\text{Rear} = (3+1)\%5 = 4$.
  - State: $\text{Front} = 1, \text{Rear} = 4$, Elements: `[_, 20, 30, 40, 50]`.
- Operation 4: `dequeue()` twice removes 20 and 30:
  - First dequeue → $\text{Front} = (1+1)\%5 = 2$.
  - Second dequeue → $\text{Front} = (2+1)\%5 = 3$.
  - State: $\text{Front} = 3, \text{Rear} = 4$, Elements: `[_, _, _, 40, 50]`.
- Operation 5: Enqueue 60, 70:
  - Enqueue 60 → $\text{Rear} = (4+1)\%5 = 0$ (Wrapped to slot 0!).
  - Enqueue 70 → $\text{Rear} = (0+1)\%5 = 1$ (Wrapped to slot 1!).
  - **Final State:** $\text{Front} = 3, \text{Rear} = 1$, Elements: `[60, 70, _, 40, 50]`.

WORKED EXAMPLE 2: IMPLEMENTING CIRCULAR QUEUE CLASS IN PYTHON

Write a Python class `CircularQueue` with fixed capacity `max_size` implementing `enqueue`, `dequeue`, and boundary checks:

```
class CircularQueue:
    def __init__(self, max_size):
        self.max = max_size
        self.queue = [None] * max_size
        self.front = -1
        self.rear = -1

    def is_empty(self):
        return self.front == -1

    def is_full(self):
        return (self.rear + 1) % self.max == self.front

    def enqueue(self, item):
        if self.is_full():
            raise OverflowError("Circular Queue Overflow")
        if self.is_empty():
            self.front = 0
            self.rear = 0
        else:
            self.rear = (self.rear + 1) % self.max
        self.queue[self.rear] = item

    def dequeue(self):
        if self.is_empty():
            raise IndexError("Circular Queue Underflow")
        val = self.queue[self.front]
        if self.front == self.rear:
            self.front = -1
            self.rear = -1
        else:
            self.front = (self.front + 1) % self.max
        return val
```

WORKED EXAMPLE 3: SIMULATING ROUND-ROBIN SCHEDULER WITH QUEUE

Given a set of 3 processes with CPU burst times: `{"P1": 8, "P2": 4, "P3": 6}` and a Round-Robin time quantum $Q = 3$ ms, simulate the order of process execution using a FIFO Queue:

Execution Trace:

- Initial Queue: `["P1:8", "P2:4", "P3:6"]`
- **Time 0–3:** Dequeue P1. Runs for 3 ms (remaining: 5). Enqueue P1 → Queue: `["P2:4", "P3:6", "P1:5"]`
- **Time 3–6:** Dequeue P2. Runs for 3 ms (remaining: 1). Enqueue P2 → Queue: `["P3:6", "P1:5", "P2:1"]`
- **Time 6–9:** Dequeue P3. Runs for 3 ms (remaining: 3). Enqueue P3 → Queue: `["P1:5", "P2:1", "P3:3"]`
- **Time 9–12:** Dequeue P1. Runs for 3 ms (remaining: 2). Enqueue P1 → Queue: `["P2:1", "P3:3", "P1:2"]`
- **Time 12–13:** Dequeue P2. Runs for 1 ms → **P2 Finished!** Queue: `["P3:3", "P1:2"]`
- **Time 13–16:** Dequeue P3. Runs for 3 ms → **P3 Finished!** Queue: `["P1:2"]`
- **Time 16–18:** Dequeue P1. Runs for 2 ms → **P1 Finished!** Queue: `[]`
- **Execution Sequence:** `P1 -> P2 -> P3 -> P1 -> P2 -> P3 -> P1`. Total completion time = 18 ms.

WORKED EXAMPLE 4: QUEUE IMPLEMENTED VIA TWO STACKS

Implement a FIFO Queue using two LIFO Stacks (`in_stack` and `out_stack`) such that both `enqueue` and `dequeue` achieve amortized $O(1)$ constant time:

```
class QueueTwoStacks:
    def __init__(self):
        self.in_stack = []
        self.out_stack = []

    def enqueue(self, item):
        self.in_stack.append(item)  # O(1)

    def dequeue(self):
        if not self.out_stack:
            # Transfer all elements from in_stack to out_stack (inverting LIFO to FIFO)
            while self.in_stack:
                self.out_stack.append(self.in_stack.pop())
        if not self.out_stack:
            raise IndexError("Queue Underflow")
        return self.out_stack.pop() # Amortized O(1)
```

Amortized Analysis: Each element is pushed to `in_stack` once, transferred to `out_stack` once, and popped from `out_stack` once. Across N operations, total work is $3N$, averaging $O(1)$ amortized cost per dequeue.

WORKED EXAMPLE 5: SLIDING WINDOW MAXIMUM USING MONOTONIC DEQUE

Given an integer array `nums = [1, 3, -1, -3, 5, 3, 6, 7]` and window size $k = 3$, calculate the maximum value in every sliding window using a Monotonic Deque in $O(N)$ linear time.

```
from collections import deque

def max_sliding_window(nums, k):
    dq = deque()  # Stores indices in decreasing order of value
    res = []

    for i in range(len(nums)):
        # Remove indices outside current window [i - k + 1, i]
        if dq and dq[0] < i - k + 1:
            dq.popleft()

        # Remove elements smaller than current element from rear
        while dq and nums[dq[-1]] < nums[i]:
            dq.pop()

        dq.append(i)

        # Start appending to result once first window is formed
        if i >= k - 1:
            res.append(nums[dq[0]])

    return res

print(max_sliding_window([1, 3, -1, -3, 5, 3, 6, 7], 3))
# Output: [3, 3, 5, 5, 6, 7]
```

WORKED EXAMPLE 6: PRIORITY QUEUE EMERGENCY TRIAGE USING `HEAPQ`

Implement a hospital emergency room triage queue where patients are served strictly by medical severity rank (1 = Critical, 2 = Urgent, 3 = Stable):

```
import heapq

class HospitalTriage:
    def __init__(self):
        self._heap = []
        self._entry_count = 0  # Tie-breaker for identical priorities

    def admit_patient(self, name, severity):
        # Tuple: (severity, entry_count, name)
        heapq.heappush(self._heap, (severity, self._entry_count, name))
        self._entry_count += 1

    def treat_next(self):
        if not self._heap:
            raise IndexError("No patients waiting.")
        severity, count, name = heapq.heappop(self._heap)
        return name, severity

triage = HospitalTriage()
triage.admit_patient("Alice", 2)
triage.admit_patient("Bob", 1)    # Critical
triage.admit_patient("Charlie", 1) # Critical (arrived second)
print(triage.treat_next())  # ('Bob', 1)
print(triage.treat_next())  # ('Charlie', 1)
print(triage.treat_next())  # ('Alice', 2)
```

Concept Module 9: Advanced Queue Architectures & Algorithmic Patterns

9.1 Circular Queue State Tracking: Sentinel Slot vs Count Variable

A classic engineering debate in circular buffer design concerns differentiating between a completely **Full** queue and a completely **Empty** queue:

- In both extremes, a basic linear pointer check finds Front == Rear.
- Two standard architectural solutions exist in production computer science:

1. The Sentinel (Sacrificial) Slot Paradigm

- The circular queue allocates an array of size Max, but caps maximum concurrent capacity at Max − 1.
- One physical slot is permanently sacrificed as a sentinel delimiter between Rear and Front.
- **Full Condition:** ((Rear + 1)% Max) == Front.
- **Empty Condition:** Front == −1 (or Front == Rear).

- **Advantage:** Highly efficient in low-level C / embedded firmware; zero secondary variable mutation.
- **Disadvantage:** Sacrifices exactly one slot of memory.

2. The Explicit Count Variable Paradigm

- An auxiliary integer counter `self.count` tracks the exact number of active elements currently in the buffer.
- When `enqueue` occurs, `count` increments by 1.
- When `dequeue` occurs, `count` decrements by 1.
- **Full Condition:** `self.count == self.max` (All Max physical slots are utilized; zero wasted memory).
- **Empty Condition:** `self.count == 0`.
- **Pointers:** Rear $=$ (Rear $+ 1$)% Max on insert; Front $=$ (Front $+ 1$)% Max on delete.
- In multi-threaded environments, maintaining an additional shared integer requires atomic updates, making the sentinel slot preferable for lock-free circular buffers.

9.2 Stream Processing & The Monotonic Deque Pattern

A **Monotonic Queue** (or Deque) is a specialized data structure that enforces strict monotonicity (either strictly non-increasing or strictly non-decreasing) among its stored elements:

- **Monotonic Decreasing Deque:** Elements are ordered from largest to smallest from Front to Rear.
  - When a new element x arrives, all smaller elements at the Rear of the deque are popped **before** x is appended.
  - **Invariant:** The front element (`dq[0]`) is always guaranteed to be the **maximum** element currently in the active sliding window.
- **Time Complexity:** Every element index is pushed once and popped at most once, enabling sliding window range maximum queries in amortized $O(1)$ time per slide, completely eliminating $O(N \times k)$ brute-force window rescans.

9.3 Producer-Consumer Concurrency & Poison-Pill Protocols

In multi-threaded server systems (such as web servers handling simultaneous client requests), queues serve as asynchronous message conduits between producer threads and worker consumer threads:

- Standard Python provides `queue.Queue(maxsize)`:
  - If a producer pushes to a full queue, `put()` automatically blocks until a consumer frees space.
  - If a consumer reads from an empty queue, `get()` automatically blocks until a producer supplies data.
- **The Poison-Pill Termination Protocol:**
  - How do background worker threads know when no more work remains?
  - Producers enqueue a sentinel object (typically `None` or a custom singleton `SENTINEL = object()`).
  - When a consumer dequeues the sentinel, it recognizes the shutdown command, breaks its infinite loop, and terminates gracefully.

9.4 Comprehensive Worked Examples (Part 2)

WORKED EXAMPLE 7: FIRST NON-REPEATING CHARACTER IN A DATA STREAM

A stream of characters arrives dynamically one by one. Using a FIFO Queue and a frequency hash table, print the first non-repeating character seen so far after every character arrival. If no non-repeating character exists, output `'#'`.
Python Implementation:

```python
from collections import deque

def first_non_repeating(stream):
    freq = {}
    q = deque()
    result = []

    for ch in stream:
        freq[ch] = freq.get(ch, 0) + 1
        q.append(ch)

        # Discard repeating characters from front of queue
        while q and freq[q[0]] > 1:
            q.popleft()

        if q:
            result.append(q[0])
        else:
            result.append('#')

    return "".join(result)

print(first_non_repeating("aabc"))  # Output: "a#bb"
```

Analysis: Characters that repeat are pruned from the Front of the queue. Each character is enqueued once and dequeued at most once, ensuring overall $O(N)$ linear processing across the entire stream.

WORKED EXAMPLE 8: LEVEL-BY-LEVEL BINARY TREE TRAVERSAL WITH LINE BREAKS

Write a Python function `print_level_order(root)` that uses a FIFO Queue to traverse a binary tree and prints all nodes residing on the same depth level together on a single line.

```
from collections import deque

class TreeNode:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def print_level_order(root):
    if not root:
        return
    q = deque([root])

    while q:
        level_size = len(q)  # Snapshot count of nodes at current depth
        level_nodes = []

        for _ in range(level_size):
            node = q.popleft()
            level_nodes.append(str(node.val))
            if node.left:
                q.append(node.left)
            if node.right:
                q.append(node.right)

        print(" ".join(level_nodes))

# Test tree:       1
#                 /   \
#                2     3
#               / \     \
#              4   5     6
root = TreeNode(1, TreeNode(2, TreeNode(4), TreeNode(5)), TreeNode(3, None, TreeNode(6)))
print_level_order(root)
# Line 1: 1
# Line 2: 2 3
# Line 3: 4 5 6
```

WORKED EXAMPLE 9: SINGLE-PRODUCER SINGLE-CONSUMER (SPSC) CIRCULAR QUEUE

Implement a circular queue using the explicit count variable strategy to utilize 100% of buffer memory without wasting a sentinel slot:

```
class CountCircularQueue:
    def __init__(self, capacity):
        self.cap = capacity
        self.buf = [None] * capacity
        self.head = 0  # Front read pointer
        self.tail = 0  # Rear write pointer
        self.count = 0

    def enqueue(self, val):
        if self.count == self.cap:
            raise OverflowError("Ring Buffer Full")
        self.buf[self.tail] = val
        self.tail = (self.tail + 1) % self.cap
        self.count += 1

    def dequeue(self):
        if self.count == 0:
            raise IndexError("Ring Buffer Empty")
        val = self.buf[self.head]
        self.head = (self.head + 1) % self.cap
        self.count -= 1
        return val
```

Master Mindmap: Queue Data Structure



Figure 3: Figure 3: Master Synthesis Mindmap: Structural framework of Unit 3 Queue Data Structures & Algorithms.

OurMocks

High-Yield Cheatsheet & Exam Traps

MASTER QUEUE ADT & DEQUE REVISION CHEATSHEET

- **FIFO Discipline:** First-In, First-Out. Insertion at Rear, deletion from Front.
- **Linear Queue Pointer Drift:** In linear queues, dequeuing moves Front rightward, stranding freed front slots. Rear = Max − 1 causes "False Overflow" despite empty front space.
- **Circular Queue Modulo Rules:** Pointers advance via ptr = (ptr + 1)% Max. Completely reuses vacated memory slots.
- **Circular Queue Full Condition:** ((Rear + 1)% Max) == Front.
- **Circular Queue Empty Condition:** Front = −1 (or Rear = −1).
- **Circular Queue Size Formula:** ((Rear − Front + Max)% Max) + 1 (for non-empty queues).
- **Double-Ended Queue (Deque):** Bi-directional operations at both Front and Rear (`append`, `appendleft`, `pop`, `popleft`) in strictly $O(1)$ time.
- **Input-Restricted Deque:** Insertion confined to **one end** (Rear); deletion allowed at **both ends** (Front and Rear).
- **Output-Restricted Deque:** Deletion confined to **one end** (Front); insertion allowed at **both ends** (Front and Rear).
- **collections.deque:** Doubly-linked memory blocks. Guaranteed strictly $O(1)$ at both extremities; standard Python tool for queues.
- **Priority Queue:** Elements dequeued based on priority key (min-heap or max-heap) rather than arrival time; $O(\log N)$ push/pop via `heapq`.

THE SEVEN FATAL CUET TRAPS IN QUEUE DATA STRUCTURES

1. **The `list.pop(0)` Linear Performance Trap:** Using `s.append(x)` and `s.pop(0)` on a Python list degrades dequeue to $O(N)$ linear time! Always utilize `collections.deque` with `popleft()` for true $O(1)$ execution.
2. **The Circular Queue Full Equation Trap:** Confusing ((Rear + 1)% Max) == Front with Rear + 1% Max == Front. Due to operator precedence, `%` binds before `+`, producing totally incorrect condition evaluations!
3. **The False Overflow Pitfall:** In linear queues, when Rear = Max − 1, the queue declares Overflow even if elements have been dequeued from the front. A circular queue is required to prevent this memory drift.
4. **Input vs Output Restriction Inversion Trap:** Remembering that **Input-Restricted** restricts **where elements enter** (insertion at 1 end, deletion at 2), while **Output-Restricted** restricts **where elements exit** (deletion at 1 end, insertion at 2).
5. **Single Element Dequeue Reset Trap:** When a queue contains exactly 1 element (Front = Rear), dequeuing that element must reset both pointers back to −1! Failing to reset leaves dangling pointers.
6. **The Deque Emulation End Mismatch Trap:** Emulating a stack with a deque requires operating on the **same end** (`append` + `pop`); emulating a queue requires operating on **opposite ends** (`append` + `popleft`).
7. **Priority Queue Key Direction Trap:** Python's `heapq` is a **Min-Heap** by default (smallest key pops first). To simulate a Max-Priority queue, priority values must be inverted (e.g. multiplied by −1).

CUET UG Exam-Style Practice Questions (15 MCQs)

Q1. Which of the following data structures operates strictly on the First-In, First-Out (FIFO) principle?

(A) Linear Queue **(B)** LIFO Stack
(C) Binary Search Tree **(D)** Priority Stack

Sol. Q1. A Queue is defined as a restricted linear data structure where elements enter at the Rear and depart from the Front, enforcing the First-In, First-Out (FIFO) access protocol.
Answer: (A)

Q2. In a circular queue of fixed capacity Max, which of the following mathematical equations correctly specifies the Full (Overflow) condition?

(A) (Front + 1)% Max == Rear **(B)** ((Rear + 1)% Max) == Front
(C) Rear == Max − 1 **(D)** (Rear − Front) == Max

Sol. Q2. In a circular ring buffer of size Max, the queue is full when advancing the Rear pointer by one position modulo Max collides with the current Front pointer: ((Rear + 1)% Max) == Front.
Answer: (B)

Q3. What is the major operational flaw of a Linear Queue implemented on a fixed-size contiguous array?

(A) Elements can only be inserted at the Front **(B)** It requires $O(N^2)$ time for peek operations
(C) It suffers from Memory Drift (False Overflow) where free slots at the front cannot be reused **(D)** It cannot store string or dictionary data types

Sol. Q3. In a linear queue, dequeuing items advances Front rightward, leaving front memory slots permanently un-reusable. When Rear reaches Max − 1, the queue reports overflow even if all front slots are empty (False Overflow).
Answer: (C)

Q4. A Deque in which insertion is strictly restricted to one logical endpoint while deletion is permitted at both endpoints is classified as:

(A) Priority Queue **(B)** Output-Restricted Deque
(C) Circular Buffer **(D)** Input-Restricted Deque

Sol. Q4. An Input-Restricted Deque is defined as a double-ended queue where insertion is restricted to one end (Rear), while deletion can be performed at both ends (Front and Rear).
Answer: (D)

Q5. What is the time complexity of dequeuing an element from a queue implemented using a standard Python list via `list.pop(0)`?

(A) $O(N)$ linear time **(B)** $O(1)$ constant time
(C) $O(\log N)$ logarithmic time **(D)** $O(N \log N)$

Sol. Q5. Because Python lists are contiguous dynamic arrays, removing the element at index 0 via `pop(0)` forces the memory controller to shift all $N - 1$ remaining elements one position to the left, taking $O(N)$ linear time.
Answer: (A)

CUET UG Exam-Style Practice Questions (Part 2)

Q6. Consider a circular queue of capacity Max = 6. Initially, Front = 2 and Rear = 5. Three elements are enqueued into the queue. What is the new value of the Rear pointer?

(A) 1 (B) 2
(C) 0 (D) 3

Sol. Q6. Modulo progression for 3 enqueues:

1. First enqueue: Rear = $(5 + 1)\%6 = 0$.
2. Second enqueue: Rear = $(0 + 1)\%6 = 1$.
3. Third enqueue: Rear = $(1 + 1)\%6 = 2$.

The new value of Rear is 2. (Notice that now Rear == Front, meaning the queue is completely full).
Answer: (B)

Q7. Which standard Python module and data structure guarantees strictly $O(1)$ worst-case time complexity for both `append()` and `popleft()` operations?

(A) `queue.LifoQueue` (B) `array.array`
(C) `collections.deque` (D) `heapq.heap`

Sol. Q7. `collections.deque` is implemented internally as a doubly-linked list of fixed-size blocks (64 elements each), providing guaranteed $O(1)$ worst-case time for insertions and deletions at both ends without dynamic array reallocation latency.
Answer: (C)

Q8. Which of the following computer systems applications fundamentally relies on a FIFO Queue data structure?

(A) Undo and Redo operation buffers in photo editors (B) Syntax parsing of balanced parentheses in compilers
(C) Backtracking in Depth-First Search (DFS) (D) Round-Robin CPU process scheduling and print spooling

Sol. Q8. Round-Robin CPU scheduling and print spoolers fundamentally rely on FIFO Queues to process tasks fairly in arrival order. Undo/Redo, delimiter parsing, and DFS all fundamentally utilize LIFO Stacks.
Answer: (D)

Q9. In a circular queue of capacity Max = 8, the pointers are currently Front = 4 and Rear = 1. How many active elements are currently residing in the queue?

(A) 6 (B) 5
(C) 3 (D) 7

Sol. Q9. Using the circular queue element count formula: Count = (Rear − Front + Max)% Max + 1 Count = $(1 - 4 + 8)\%8 + 1 = (5)\%8 + 1 = 5 + 1 = 6$. The active elements occupy slots $4, 5, 6, 7, 0, 1$ (exactly 6 items).
Answer: (A)

Q10. In an Output-Restricted Deque, which operations are permitted?

(A) Insertion at both ends, deletion at both ends (B) Insertion at both ends, deletion restricted to one end
(C) Deletion at both ends, insertion restricted to one end (D) Insertion and deletion restricted to the same single end

Sol. Q10. An Output-Restricted Deque is defined as one where output (deletion) is restricted strictly to one end (Front), while input (insertion) is permitted freely at both ends (Front and Rear).
Answer: (B)

CUET UG Exam-Style Practice Questions (Part 3)

Q11. What is the worst-case time complexity of inserting an element into and extracting the highest priority element from a Priority Queue backed by a Binary Heap?

(A) Insert: $O(1)$, Extract: $O(N)$
(B) Insert: $O(N)$, Extract: $O(1)$
(C) Insert: $O(\log N)$, Extract: $O(\log N)$
(D) Insert: $O(1)$, Extract: $O(1)$

Sol. Q11. In a Binary Heap implementation of a Priority Queue, inserting an element requires heapifying up ($O(\log N)$), and extracting the root element requires heapifying down ($O(\log N)$).
Answer: (C)

Q12. When a queue is implemented using two LIFO stacks (`in_stack` and `out_stack`), what is the amortized time complexity of each `dequeue()` operation?

(A) $O(N^2)$ quadratic time
(B) $O(N)$ linear time
(C) $O(\log N)$ logarithmic time
(D) $O(1)$ constant amortized time

Sol. Q12. Although transferring items from `in_stack` to `out_stack` takes $O(N)$ time when `out_stack` is empty, each item is moved at most once across its lifecycle. Amortized across N dequeues, the cost per operation is strictly $O(1)$ constant time.
Answer: (D)

Q13. Consider an initially empty circular queue of capacity 4. The following operations are executed: `enqueue(A)`, `enqueue(B)`, `dequeue()`, `enqueue(C)`, `enqueue(D)`, `enqueue(E)`. If an immediate subsequent operation `enqueue(F)` is attempted, what occurs?

(A) Queue Overflow error is raised
(B) Element B is overwritten by F
(C) Queue underflow occurs
(D) F is successfully enqueued at slot 1

Sol. Q13. Tracing state:

1. Enqueue A → `Front=0, Rear=0 [A, _, _, _]`
2. Enqueue B → `Front=0, Rear=1 [A, B, _, _]`
3. Dequeue() → removes A, `Front=1, Rear=1 [_, B, _, _]`
4. Enqueue C → `Front=1, Rear=2 [_, B, C, _]`
5. Enqueue D → `Front=1, Rear=3 [_, B, C, D]`
6. Enqueue E → wraps to slot 0, `Rear=(3+1)%4=0`. Queue has 4 items: `[E, B, C, D]`.

Now the queue is completely full because `(Rear + 1) % 4 = (0 + 1) % 4 = 1 == Front`. Attempting `enqueue(F)` triggers `Queue Overflow`!
Answer: (A)

Q14. Which graph traversal algorithm fundamentally utilizes a Queue data structure to explore nodes in order of their distance from the source vertex?

(A) Depth-First Search (DFS)
(B) Breadth-First Search (BFS)
(C) Topological Sort using DFS
(D) Backtracking Search

Sol. Q14. Breadth-First Search (BFS) fundamentally relies on a FIFO Queue to visit all immediate neighbor nodes at distance k before proceeding to explore vertices at distance $k+1$.
Answer: (B)

Q15. In a circular queue, when the last remaining element is removed via `dequeue()`, what must the `front` and `rear` pointers be reset to?

(A) `front = 0, rear = 0`
(B) `front = 0, rear = -1`
(C) `front = -1, rear = -1`
(D) `front = Max - 1, rear = Max - 1`

Sol. Q15. When the solitary remaining element in a circular queue is dequeued (Front == Rear), both pointers must be reset to -1 (`front = -1, rear = -1`) to represent the empty queue state.
Answer: (C)